Roteiro de fala

Serverless no Google Cloud: Cloud Run e Cloud Run functions

Este roteiro acompanha os 10 slides e soma cerca de 13 minutos. As falas são uma base para dizer com as suas palavras; não é preciso decorar. Os trechos entre colchetes, em destaque, são ações: o que mostrar, apontar ou esperar. As mesmas falas também estão nas notas do apresentador de cada slide.

Visão geral do tempo

Slide	Assunto	Tempo
1	Abertura	15 s
2	Qual problema o serverless resolve	1 min
3	Carro próprio ou aplicativo?	1 min 30 s
4	Dois serviços, um mesmo motor	1 min
5	O ciclo em 4 passos	1 min
6	Arquitetura da Loja Exemplo	3 min
7	Vantagens e limitações	1 min
8	Serverless ou VMs com autoscaling?	1 min
9	Demonstração	2 min
10	Fechamento	1 min

Falas, slide a slide

Slide 1 · Abertura

Tempo: 15 s

Boa tarde. Nosso tema é serverless no Google Cloud, com dois serviços: o Cloud Run e o Cloud Run functions.

A ideia é que, ao final, vocês saibam quando usar cada um e também quando não usar nenhum dos dois. Vamos responder cinco perguntas: qual problema o serviço resolve, como funciona, como fica numa arquitetura, quais as vantagens e limitações, e no fim uma demonstração ao vivo.

Slide 2 · Qual problema o serverless resolve

Tempo: 1 min

Imagine uma loja online. De madrugada quase ninguém acessa; na Black Friday, milhares de pessoas entram ao mesmo tempo.

Com um servidor tradicional, a empresa precisa escolher: ou compra uma máquina grande e paga por ela parada o ano todo, ou compra uma pequena e o site cai no pico. E tem mais: alguém precisa atualizar o sistema operacional, aplicar correções de segurança e monitorar essas máquinas.

Então são dois problemas: pagar por servidor parado e gastar tempo cuidando de infraestrutura em vez de cuidar do produto.

Slide 3 · Carro próprio ou aplicativo?

Tempo: 1 min 30 s

Uma forma simples de enxergar isso: ter servidor próprio é como ter carro próprio. Você paga IPVA, seguro e manutenção mesmo com ele parado na garagem.

Serverless é como usar um aplicativo de transporte: você paga só pela corrida, e a manutenção do carro não é problema seu.

E essa comparação também vale para o custo. Se você usa o carro o dia inteiro, todos os dias, ter um próprio sai mais barato. Mas se precisa dele só uma vez por semana, compensa muito mais chamar pelo aplicativo do que comprar um. Com serverless é igual: quanto mais o serviço fica parado, mais você economiza; se ele é usado muito e o tempo todo, um servidor fixo pode sair mais barato.

Um cuidado com o nome: serverless não significa sem servidor. Os servidores existem, mas são do Google, e é ele quem cuida deles.

Resumindo: você entrega o código, o Google cuida do resto, e você paga só quando ele roda.

Slide 4 · Dois serviços, um mesmo motor

Tempo: 1 min

No Google Cloud, isso aparece em dois serviços que funcionam da mesma forma por baixo. A diferença é o tamanho do que você entrega.

No Cloud Run, você entrega uma aplicação inteira empacotada em um container: uma API, um site com várias rotas.

No Cloud Run functions, você entrega só uma função: um pedaço pequeno de código que faz uma única tarefa e é disparado por um evento, como um arquivo enviado ou uma mensagem que chegou.

Uma curiosidade: até 2024 esse segundo serviço se chamava Cloud Functions. Mudou de nome justamente porque passou a rodar dentro do Cloud Run. Se vocês pesquisarem e acharem o nome antigo, é o mesmo serviço.

A regra prática é: aplicação inteira vai no Cloud Run; tarefa pequena que reage a um evento vai numa function.

Slide 5 · O ciclo em 4 passos

Tempo: 1 min

Como isso funciona na prática? São quatro passos.

Primeiro, chega uma requisição ou um evento. Segundo, se não houver nenhuma instância ligada, o Google liga uma. Essa espera inicial tem nome: cold start. Voltando à analogia, é o tempo do motorista chegar até você.

Terceiro, o código executa. Se chegarem muitos pedidos, o Google liga mais instâncias sozinho. Repare que ele cresce para os lados: em vez de aumentar uma máquina, liga mais instâncias pequenas. Quarto, quando os pedidos param, as instâncias são desligadas até zero, e a cobrança para junto.

Slide 6 · Arquitetura da Loja Exemplo

Tempo: 3 min

Para ver isso numa arquitetura real, criamos uma loja fictícia, a Loja Exemplo. As duas caixas em destaque são os serviços do nosso tema; o resto é o que existe ao redor deles em produção. Vamos ler o desenho por três caminhos.

[Apontar a linha de cima e o Cloud Run] Primeiro, o cliente navegando. Ele acessa loja-exemplo.com. O Cloud DNS aponta o domínio para o Load Balancer, que cuida do HTTPS, bloqueia ataques com o Cloud Armor e entrega conteúdo repetido pela CDN. O restante chega ao Cloud Run, que monta as páginas e responde à API consultando o banco no Cloud SQL.

[Apontar a linha de baixo] Segundo, o lojista cadastrando uma foto. O Cloud Run grava a foto no bucket de originais. O Eventarc percebe o arquivo novo e dispara a function, que gera a

miniatura e salva em outro bucket. De lá, a CDN entrega as miniaturas direto, sem passar pelo Cloud Run. Repare que ninguém mandou a function rodar: ela reagiu a um evento, a foto chegando.

[Apontar o rodapé] Terceiro, a equipe operando. O Cloud Build gera a imagem e publica a nova versão, a senha do banco fica no Secret Manager, e logs e métricas de tudo vão para o Cloud Logging e Monitoring.

Um detalhe importante: nem tudo aqui escala a zero. O Cloud SQL fica ligado o tempo todo e é cobrado por hora. O serverless está na camada de computação.

E por que dividir assim? O site é uma aplicação inteira, então vai no Cloud Run. Gerar miniatura é uma tarefa pequena e isolada, então vai numa function. E se a geração de miniaturas falhar, o site continua no ar.

Slide 7 · Vantagens e limitações

Tempo: 1 min

Vantagens: paga só pelo uso e vai a zero quando ninguém acessa; escala sozinho no pico, em segundos; e não tem manutenção de servidor.

Limitações: o cold start deixa o primeiro acesso, depois de um tempo parado, mais lento; e uso alto e constante pode sair mais caro que servidor fixo.

E lembrem do carro: uso alto e constante pode sair mais caro do que um servidor fixo.

Um bom caso: site de inscrição de vestibular, parado o ano todo e lotado em uma semana. Um mau caso: transmissão de vídeo ao vivo rodando 24 horas.

Slide 8 · Serverless ou VMs com autoscaling?

Tempo: 1 min

Talvez vocês estejam pensando: VM com autoscaling e load balancer também cria máquinas novas quando satura. É verdade, a ideia de escalar é a mesma. O que muda é o tamanho da unidade, a velocidade e quem faz o trabalho.

Com VMs, a unidade é uma máquina inteira, que leva minutos para subir, e normalmente fica ao menos uma ligada o tempo todo. Você paga por VM ligada e precisa manter imagem, patches, template, regras de escala e o load balancer.

No Cloud Run, a unidade é um container, que sobe em segundos e pode ir a zero. Você paga pelo uso e cuida só do código. Em troca, as VMs dão controle total do ambiente.

Slide 9 · Demonstração

Tempo: 2 min

Agora vamos ver funcionando.

[Mostrar o código] São cinco linhas. Não tem servidor, não tem configuração de máquina. Só a função.

[Abrir a URL com ?nome=Professor] Isso está rodando na nuvem do Google agora.

[Disparar a carga com hey ou um loop de curl] Agora vou simular a Black Friday.

[Abrir o gráfico de instâncias e apontar a subida] Ninguém configurou isso. O Google percebeu a demanda e ligou mais instâncias. Daqui a alguns minutos, sem acesso, isso volta para zero, e a conta para de correr.

Slide 10 · Fechamento

Tempo: 1 min

Para fechar, serverless é pagar só pela corrida. Aplicação inteira vai no Cloud Run; tarefa pequena que reage a um evento vai numa function.

Obrigado! Ficamos abertos a perguntas.

Antes da apresentação: preparar a demo

Publique a função com antecedência e deixe abertas, em abas, a URL da função e o painel de métricas do Cloud Run com o gráfico de instâncias.

Código (main.py):

```
import functions_framework

@functions_framework.http
def ola(request):
    nome = request.args.get("nome", "turma")
    return f"Olá, {nome}!"
```

Comando de publicação:

```
gcloud functions deploy ola --gen2 --runtime=python312 \
  --region=southamerica-east1 --trigger-http \
  --allow-unauthenticated --entry-point=ola
```

Plano B: grave a demo inteira em vídeo antes. Se a internet da sala falhar, apresente o vídeo narrando o mesmo roteiro.

Perguntas que a plateia pode fazer

VM com autoscaling não faz a mesma coisa?
A ideia de escalar é a mesma. A diferença é que a VM sobe em minutos, normalmente fica ao menos uma ligada e a equipe mantém tudo; no Cloud Run a instância sobe em segundos, pode ir a zero e o Google cuida da infraestrutura.

Dá para evitar o cold start?
Dá para reduzir configurando instâncias mínimas, que ficam sempre ligadas. Em troca, elas são cobradas mesmo sem tráfego.

E se passar de 100 instâncias?
Esse é só o limite padrão e pode ser aumentado. Se todas as instâncias estiverem no limite, as novas requisições esperam em fila por pouco tempo e depois falham.

Por que não usar uma máquina maior?
Uma máquina maior aumenta a capacidade, mas continua sendo um único ponto de falha. Várias instâncias pequenas sobem mais rápido e, se uma cair, as outras continuam atendendo.

Glossário rápido

Termo	Em uma frase
Serverless	Modelo em que o provedor cuida dos servidores e você paga só pelo uso
Instância	Uma cópia do seu código rodando; o Google liga e desliga quantas forem necessárias
Evento	Algo que acontece e dispara a função, como um upload ou um acesso a um link

Cold start	Atraso no primeiro acesso, enquanto uma instância nova é ligada
Escalar a zero	Desligar todas as instâncias quando não há uso, zerando o custo

Informações extras para estudo

Não fazem parte da apresentação. Servem para você entender melhor o funcionamento e responder caso alguém pergunte.

Quando o Google liga uma nova instância

Mesmo sem configurar nada, todo serviço do Cloud Run nasce com valores padrão: cada instância tem 1 vCPU e 512 MiB de memória, atende até 80 requisições ao mesmo tempo, e o serviço pode chegar a 100 instâncias. Todos esses valores podem ser alterados.

Uma nova instância sobe quando as atuais passam de cerca de 60% de uso de CPU ou de 60% das requisições simultâneas que aguentam, numa média de um minuto. Por isso não existe um número fixo de requisições: uma API leve, que passa a maior parte do tempo esperando o banco, aguenta dezenas de requisições por segundo em cada instância; uma tarefa pesada, que usa muita CPU, só algumas.

O Google cresce para os lados, adicionando instâncias, e não para cima, aumentando uma máquina. Instâncias pequenas sobem em segundos, não têm teto de tamanho e, se uma cair, as outras continuam atendendo. Uma máquina maior aumentaria a capacidade, mas continuaria sendo um único ponto de falha.

Resumo: *você decide o tamanho de cada instância; o Google decide quantas ligar.*

Quanto custa na prática

Estimativa só da camada de computação de uma API, com preços da região us-central1. Os valores são aproximados e servem apenas para ter uma noção.

Cenário	Tráfego por mês	Cloud Run	VMs com autoscaling
Loja pequena	500 mil requisições	≈ US$ 0	≈ US$ 49
Loja média	5 milhões de requisições	≈ US$ 21	≈ US$ 49
Loja grande, tráfego constante	50 milhões de requisições	≈ US$ 266	≈ US$ 147 (≈ US$ 92 com desconto de 1 ano)

Premissas: no Cloud Run, instância de 1 vCPU e 512 MiB, 200 ms por requisição, descontada a camada gratuita mensal, sem considerar concorrência (por isso o valor real tende a ser menor). Nas VMs, e2-medium a cerca de US$ 24,46 por mês, com no mínimo 2 ligadas. Banco, armazenamento, rede e load balancer ficam fora da conta.

Lição: serverless economiza quando há ociosidade. Quanto mais alto e constante o tráfego, mais um servidor fixo passa a compensar.

O que é o Eventarc

O Eventarc é o serviço do Google Cloud que leva eventos de onde eles acontecem até quem precisa reagir a eles. Na arquitetura da Loja Exemplo, é ele que percebe que uma foto chegou no bucket fotos-originais e dispara a function que gera a miniatura.

As três peças:

Origem: onde o evento acontece. Por exemplo, Cloud Storage, Pub/Sub ou logs de auditoria de outros serviços. Aplicações também podem enviar eventos próprios por um tópico do Pub/Sub.

Gatilho (trigger): a regra que diz quais eventos interessam, filtrando pelos atributos do evento. Na loja: arquivo novo salvo no bucket fotos-originais.

Destino: quem recebe o evento: Cloud Run, Cloud Run functions, Workflows, GKE ou um endpoint HTTP.

Os eventos chegam num formato padrão chamado CloudEvents. Por baixo, o Eventarc usa o Pub/Sub como transporte, criando e gerenciando sozinho um tópico e uma assinatura. Por isso a entrega é do tipo "pelo menos uma vez": o mesmo evento pode chegar duas vezes, e a função deve aguentar repetição sem causar problema.

A grande vantagem é o desacoplamento. O site no Cloud Run não sabe que a function existe; ele só salva a foto. Para adicionar uma segunda reação ao mesmo upload, como verificar se a imagem é imprópria, basta criar outro gatilho, sem mudar o site. Quando se publica uma function disparada por evento do Storage, o gatilho do Eventarc é criado junto.